LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Modul 6: Protokol Komunikasi Telemetri Industri (MQTT)

Disusun oleh :

Nama : syafa sartika putri aryanti

NPM : 25782056

Kelas : TRI-3B

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

6.6 Latihan

- **Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):** Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:
  i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.
  ii. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.
  iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

Jawab :

i. Efisiensi Penggunaan Kapasitas *Bandwidth* pada Paket Data Seluler:

- **HTTP:** Menggunakan *header* teks yang sangat besar (bisa mencapai ratusan *byte* per permintaan) dan mengharuskan pembukaan/penutupan koneksi TCP baru setiap kali data dikirim (*overhead* jaringan tinggi).
- **MQTT:** Menggunakan *header* biner yang sangat ringan (hanya 2 *byte*) dan menjaga koneksi TCP tetap terbuka (*persistent connection*). Hal ini menghemat penggunaan data seluler hingga lebih dari 80-90% dibandingkan HTTP.

ii. Beban Komputasi Mikrokontroler (*CPU Overhead*) saat Menangani Pertukaran Data Dua Arah:

- **HTTP:** Mikrokontroler harus secara aktif melakukan *polling* (bertanya terus-menerus ke server) untuk mengecek apakah ada perintah kontrol baru, yang membuang daya pemrosesan CPU dan siklus daya.
- **MQTT:** Berbasis *event-driven* (Publish/Subscribe). Mikrokontroler bersifat pasif mendengarkan topik; saat ada data masuk, broker langsung mendorong (*push*) data tersebut ke perangkat melalui fungsi *callback*, sehingga beban pemrosesan CPU menjadi jauh lebih ringan.

iii. Kemampuan Pengiriman Data Secara Simultan ke Banyak Perangkat Penerima (*1-to-N Distribution*):

- **HTTP:** Bersifat *point-to-point* (1-to-1). Jika data ingin dikirim ke *N* perangkat, server harus melakukan koneksi HTTP terpisah sebanyak *N* kali.
- **MQTT:** Memiliki arsitektur terpusat via *Broker*. Satu pengirim (*Publisher*) cukup menerbitkan paket data satu kali ke satu topik, dan *Broker* MQTT yang akan mereplikasi serta mendistribusikannya ke seluruh *N* perangkat (*Subscribers*) secara simultan.

- **Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):** Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi

topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

i. Memantau parameter suhu dari seluruh mesin di semua lini produksi.
ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.
iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

Jawab :

Memantau parameter suhu dari seluruh mesin di semua lini produksi:

- Ekspresi Topik: **pabrik/+/+/suhu**
- *Penjelasan:* Karakter *single-level wildcard* (+) digunakan untuk menggantikan tingkat hirarki lini (<line>) dan mesin (<mesin>) yang dinamis, sehingga hanya menyaring data yang berujung pada parameter suhu dari seluruh kombinasi lini dan mesin.

Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2:

- Ekspresi Topik: **pabrik/line2/m1/+** *(atau dapat juga menggunakan **pabrik/line2/m1/#**)*
- *Penjelasan:* Jalur hierarki untuk lini dan mesin telah ditentukan secara spesifik (line2 dan m1), sedangkan level terakhir (<sensor>) menggunakan karakter *wildcard* (+) untuk mencakup semua jenis metrik sensor yang dilaporkan oleh mesin tersebut secara bersamaan.

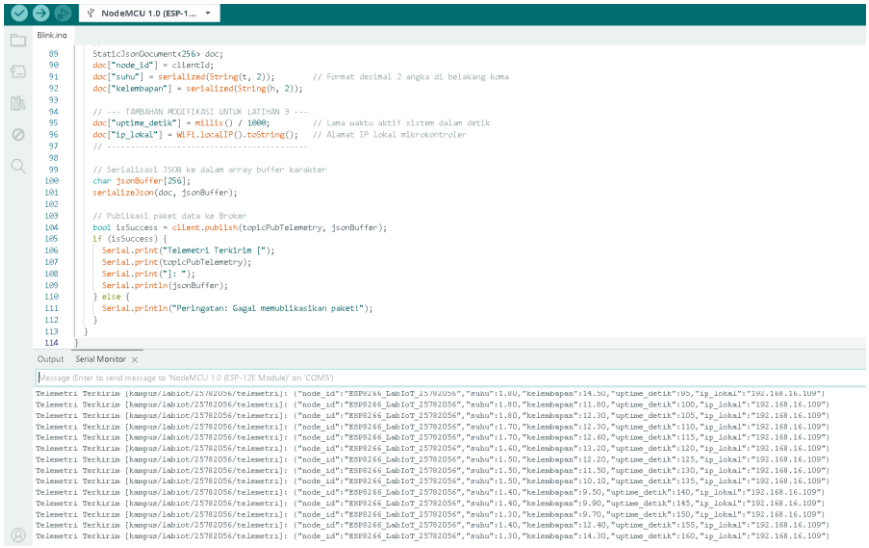
Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut:

- Ekspresi Topik: **pabrik/#**
- *Penjelasan:* Karakter *multi-level wildcard* (#) digunakan untuk mencakup dan mendengarkan seluruh tingkat sub-topik apa pun yang berada di bawah cabang utama pabrik, baik itu menyangkut seluruh lini, semua mesin, hingga beragam jenis sensor di dalamnya.

- **Latihan 3 (Implementasi Serialisasi Data Dinamis):** Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:
  i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
  ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
  Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

Jawab :

Keluaran hasil modifikasi :



Kode modifikasi :

Blink.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "syafa";
const char* password = "apayaaaaa";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// IDENTITAS KLIEN (Disesuaikan dengan NIM: 25782056)
const char* clientId = "ESP8266_LabIoT_25782056";

// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/25782056/telemetri";

// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2;   // Pin D4 (GPIO 2)
#define DHTTYPE DHT11

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long previousPublishTime = 0;
const unsigned long publishInterval = 5000; // Interval pengiriman 5 detik

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("Alamat IP Lokal: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Menghubungkan ke MQTT Broker...");
    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
```

Blink.ino

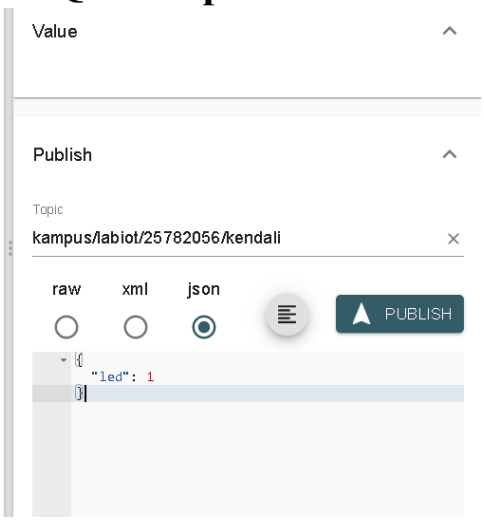
```
46  void reconnectMQTT() {
47    while (!client.connected()) {
48      Serial.print("Menghubungkan ke MQTT Broker...");
49      if (client.connect(clientId)) {
50        Serial.println(" Sukses Terhubung!");
51      } else {
52        Serial.print(" Gagal, kode status rc=");
53        Serial.print(client.state());
54        Serial.println(". Mencoba kembali dalam 5 detik...");
55        delay(5000);
56      }
57    }
58  }
59
60  void setup() {
61    Serial.begin(115200);
62    dht.begin();
63
64    connectToWiFi();
65
66    client.setServer(mqttServer, mqttPort);
67  }
68
69  void loop() {
70    if (!client.connected()) {
71      reconnectMQTT();
72    }
73    client.loop(); // Memelihara sesi TCP dan keep-alive ping
74
75    unsigned long currentMillis = millis();
76    if (currentMillis - previousPublishTime >= publishInterval) {
77      previousPublishTime = currentMillis;
78
79      float t = dht.readTemperature();
80      float h = dht.readHumidity();
81
82      // Proteksi pembacaan sensor DHT
83      if (isnan(t) || isnan(h)) {
84        Serial.println("Peringatan: Gagal membaca data DHT!");
85        return;
86      }
87
88      // Alokasi dokumen JSON statis
89      StaticJsonDocument<256> doc;
90      doc["node_id"] = clientId;
91      doc["suhu"] = serialized(String(t, 2));        // Format desimal 2 angka di belakang koma
92      doc["kelembapan"] = serialized(String(h, 2));
93
94      // --- TAMBAHAN MODIFIKASI UNTUK LATIHAN 3 ---
95      doc["uptime_detik"] = millis() / 1000;         // Lama waktu aktif sistem dalam detik
```

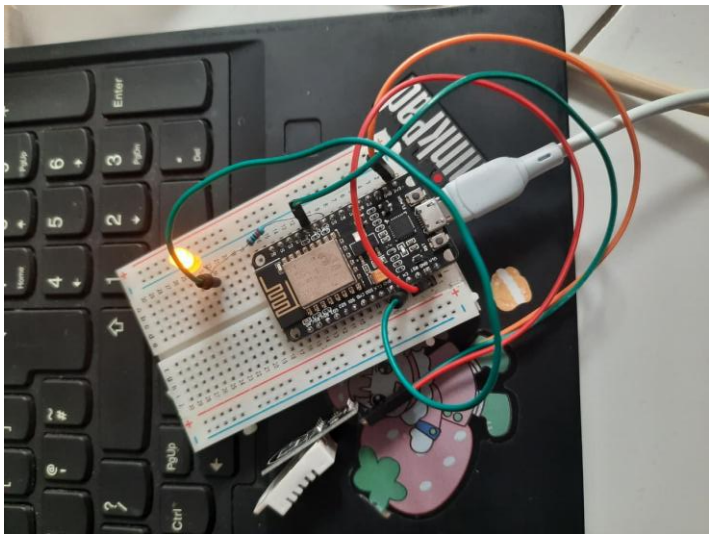
```
    // --- TAMBAHAN MODIFIKASI UNTUK LATIHAN 3 ---
    doc["uptime_detik"] = millis() / 1000;          // Lama waktu aktif sistem dalam detik
    doc["ip_lokal"] = WiFi.localIP().toString();    // Alamat IP lokal mikrokontroler
    // -------------------------------------------

    // Serialisasi JSON ke dalam array buffer karakter
    char jsonBuffer[256];
    serializeJson(doc, jsonBuffer);

    // Publikasi paket data ke Broker
    bool isSuccess = client.publish(topicPubTelemetry, jsonBuffer);
    if (isSuccess) {
      Serial.print("Telemetri Terkirim [");
      Serial.print(topicPubTelemetry);
      Serial.print("]: ");
      Serial.println(jsonBuffer);
    } else {
      Serial.println("Peringatan: Gagal memublikasikan paket!");
    }
  }
```

MQTT Explorer:





Kesimpulannya :

- Pengayaan Payload Telemetri:
  Blok penyusunan JSON pada paket telemetri MQTT berhasil dimodifikasi secara dinamis dengan menyertakan parameter tambahan uptime_detik (untuk mencatat lama waktu aktif mikrokontroler menggunakan fungsi millis()) dan ip_lokal (untuk mengidentifikasi alamat IP perangkat di jaringan lokal menggunakan WiFi.localIP().toString()).
- Validasi Kinerja Sistem: Berdasarkan hasil pengujian pada Serial Monitor, parameter tambahan tersebut berhasil diserialisasikan ke dalam format string JSON yang valid dan terkirim secara periodik ke *broker* MQTT (broker.hivemq.com) melalui topik unik kampus/labiot/25782056/telemetri.
- Manfaat Teknis:

Penambahan data dinamis ini sangat efektif untuk pemantauan kesehatan perangkat (*device health monitoring*) secara *real-time*, memungkinkan pengembang mendeteksi stabilitas sistem, durasi kerja alat, serta melacak status konektivitas jaringan perangkat mikrokontroler dari jarak jauh.

- **Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking): Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().**
  - **i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?**
  - **ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!**

Jawab :

1. Konsekuensi Teknis Penyisipan delay(6000) di Dalam Fungsi loop():

- Gangguan pada *Callback* (Kehilangan Responsivitas): Fungsi client.loop() berfungsi untuk mendengarkan, memproses, dan memicu fungsi *callback* saat ada pesan atau perintah kontrol masuk dari *broker*. Jika disisipkan delay(6000) (jeda 6 detik), eksekusi program di dalam loop() akan terhenti total (*blocking*) selama 6 detik tersebut, sehingga NodeMCU tidak akan merespons perintah kontrol secara *real-time*.
- Putus Koneksi Akibat *Keep-Alive Timeout*: Protokol MQTT memiliki mekanisme *Keep-Alive* (pengiriman paket *ping* secara berkala ke *broker*). Jika fungsi client.loop() tertunda oleh perintah delay yang melebihi batas waktu toleransi (*timeout*), *broker* MQTT akan menganggap perangkat mikrokontroler telah mati atau *offline*, sehingga *broker* akan memutuskan (*disconnect*) koneksi TCP secara sepihak.

2. Cara Kerja Fungsi reconnect() Memulihkan Koneksi saat Terputus Sesaat:

- Pemeriksaan Status Koneksi (!client.connected()): Di awal fungsi loop(), program secara kontinu mengecek apakah koneksi ke *broker* MQTT masih aktif. Jika terdeteksi terputus, fungsi reconnect() akan dipanggil secara otomatis.
- Eksekusi Blok Percobaan Hubung Ulang (client.connect): Fungsi reconnect() menggunakan struktur pengulangan (while) untuk mencoba menghubungkan kembali perangkat ke *broker* MQTT menggunakan identitas klien (clientId) yang sama.
- Pendaftaran Ulang Topik (*Re-subscribing*): Setelah koneksi TCP dan sesi MQTT berhasil dibangun kembali oleh fungsi *reconnect*, mikrokontroler akan otomatis mendaftarkan ulang subskripsi topik (misalnya kampus/labiot/25782056/kendali) agar perangkat dapat kembali menerima perintah kontrol dari jarak jauh tanpa harus melakukan *restart* manual perangkat keras.